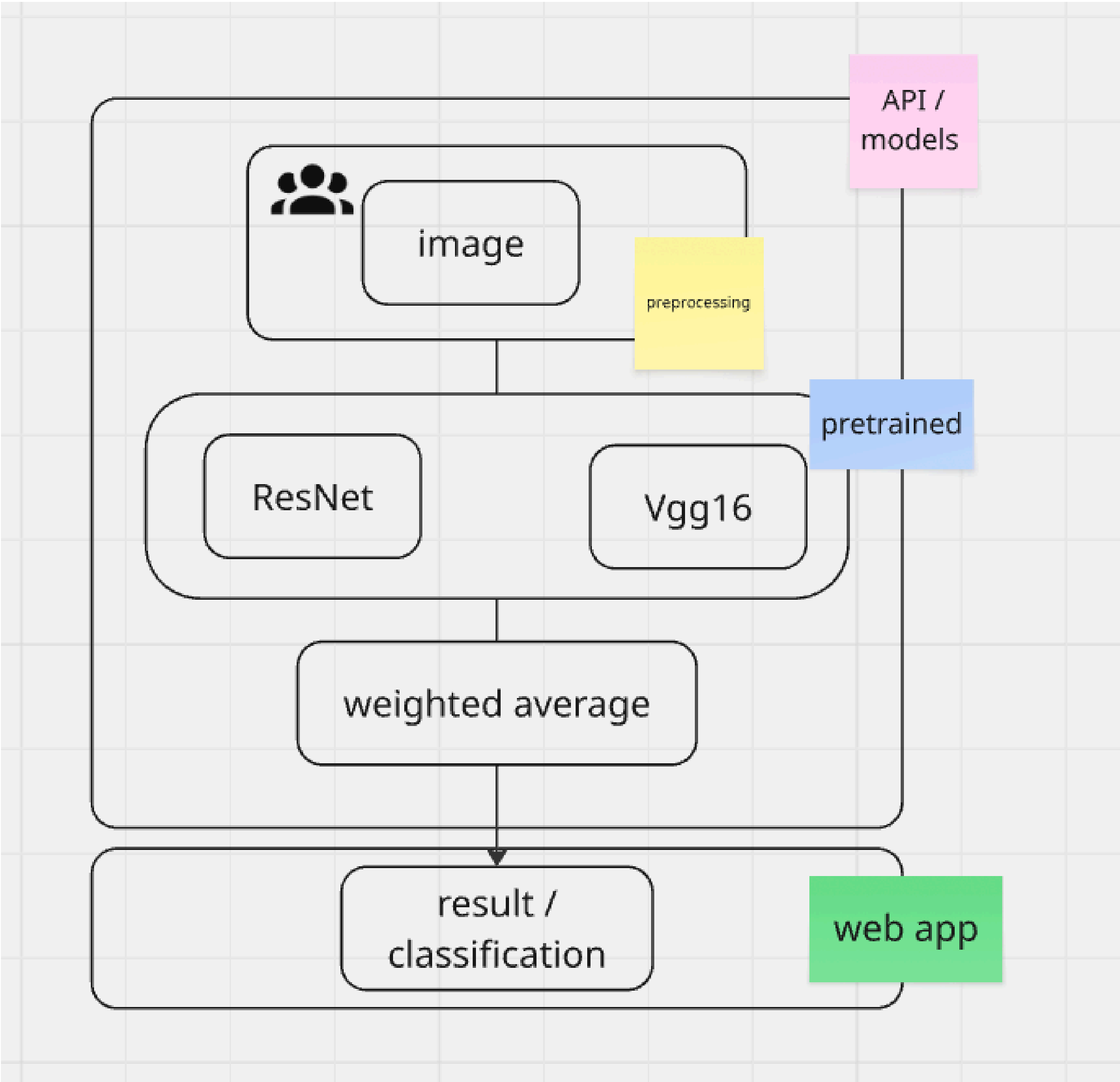
GARBAGE CLASSIFIER

Workflow :



Initial data :

```
# total images in each category
for category in os.listdir(sub_root):
  category_path = os.path.join(sub_root,category)
  print(f'{category} : {len(os.listdir(category_path))}')
```

```
cardboard : 403
plastic : 482
metal : 410
glass : 501
trash : 137
paper : 594
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    sub_root,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    sub_root,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

# Function to duplicate the image tensor for the dual-input model
def format_example(image, label):
  return ({'resnet_input': image, 'vgg_input': image}, label)

train_ds = train_ds.map(format_example).cache().prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds = val_ds.map(format_example).cache().prefetch(buffer_size=tf.data.AUTOTUNE)
```

```
Found 2527 files belonging to 6 classes.
Using 2022 files for training.
Found 2527 files belonging to 6 classes.
```

RESULTS

Without transfer learning :

```python
# without transfer learning
from tensorflow import keras
from tensorflow.keras import layers , models

number_classes = 6
model = models.Sequential([
    layers.Rescaling(1./255,input_shape=(224,224,3)),
    layers.Conv2D(16,3,padding='same',activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32,3,padding='same',activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64,3,padding='same',activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),
    layers.Dense(64,activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(32,activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(number_classes)
    ])
```

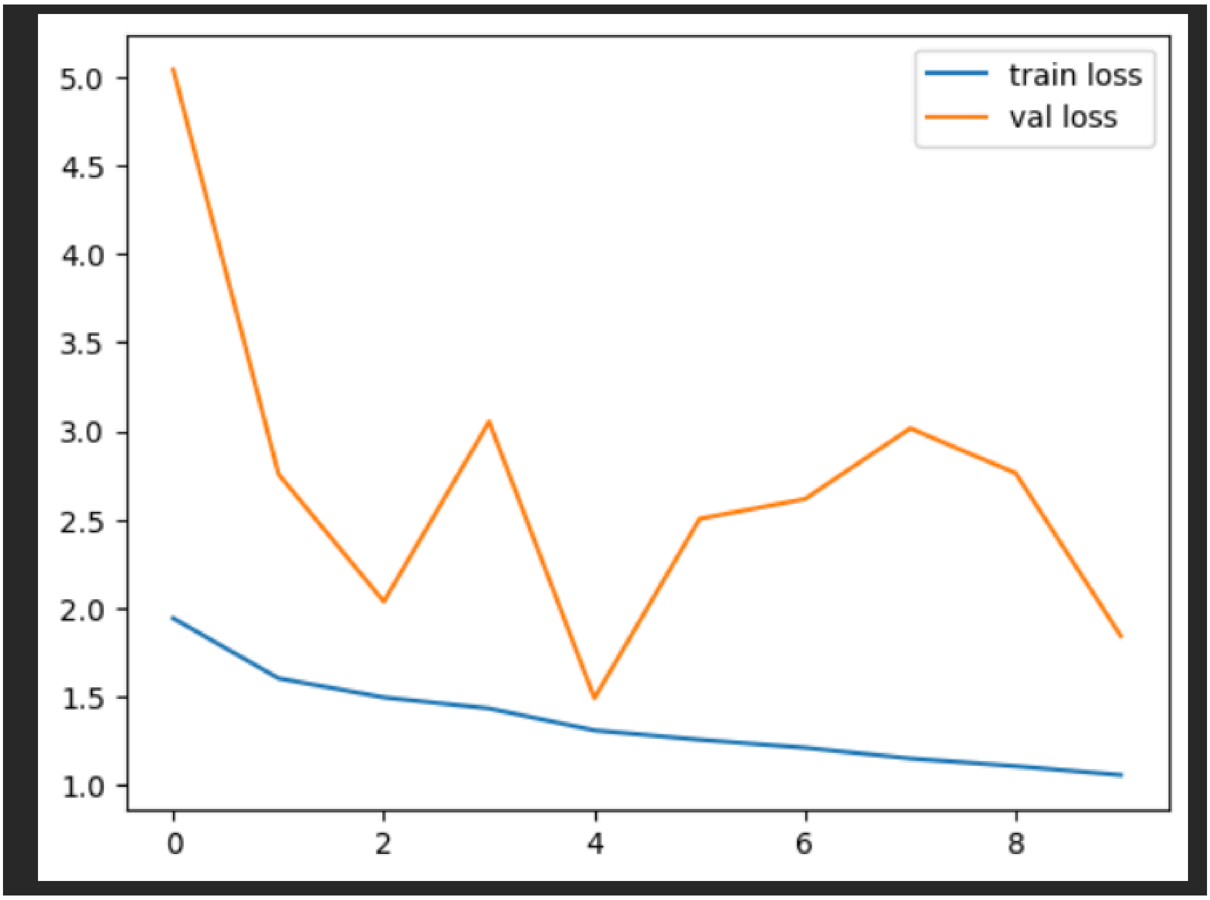
Model: "sequential_3"

Layer (type)	Output Shape	Param #
rescaling_3 (Rescaling)	(None, 224, 224, 3)	0
conv2d_9 (Conv2D)	(None, 224, 224, 16)	448
max_pooling2d_9 (MaxPooling2D)	(None, 112, 112, 16)	0
conv2d_10 (Conv2D)	(None, 112, 112, 32)	4,640
max_pooling2d_10 (MaxPooling2D)	(None, 56, 56, 32)	0
conv2d_11 (Conv2D)	(None, 56, 56, 64)	18,496
max_pooling2d_11 (MaxPooling2D)	(None, 28, 28, 64)	0
flatten_3 (Flatten)	(None, 50176)	0
dense_9 (Dense)	(None, 128)	6,422,656
batch_normalization_5 (BatchNormalization)	(None, 128)	512
dropout_5 (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 64)	8,256
batch_normalization_6 (BatchNormalization)	(None, 64)	256
dropout_6 (Dropout)	(None, 64)	0
dense_11 (Dense)	(None, 32)	2,080
batch_normalization_7 (BatchNormalization)	(None, 32)	128
dropout_7 (Dropout)	(None, 32)	0
dense_12 (Dense)	(None, 6)	198

Total params: 6,457,670 (24.63 MB)
Trainable params: 6,457,222 (24.63 MB)
Non-trainable params: 448 (1.75 KB)

```
Epoch 1/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 14s 126ms/step - accuracy: 0.3071 - loss: 1.9418 - val_accuracy: 0.2455 - val_loss: 5.0418
Epoch 2/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 3s 45ms/step - accuracy: 0.4011 - loss: 1.6027 - val_accuracy: 0.3069 - val_loss: 2.7560
Epoch 3/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 6s 57ms/step - accuracy: 0.4402 - loss: 1.4951 - val_accuracy: 0.3703 - val_loss: 2.0362
Epoch 4/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 4s 44ms/step - accuracy: 0.4679 - loss: 1.4321 - val_accuracy: 0.1802 - val_loss: 3.0532
Epoch 5/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 5s 46ms/step - accuracy: 0.4960 - loss: 1.3085 - val_accuracy: 0.4535 - val_loss: 1.4911
Epoch 6/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 5s 71ms/step - accuracy: 0.5366 - loss: 1.2558 - val_accuracy: 0.2653 - val_loss: 2.5042
Epoch 7/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 3s 45ms/step - accuracy: 0.5455 - loss: 1.2104 - val_accuracy: 0.1980 - val_loss: 2.6158
Epoch 8/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 5s 45ms/step - accuracy: 0.5732 - loss: 1.1496 - val_accuracy: 0.1960 - val_loss: 3.0152
Epoch 9/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 6s 51ms/step - accuracy: 0.5870 - loss: 1.1065 - val_accuracy: 0.2614 - val_loss: 2.7620
Epoch 10/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 5s 44ms/step - accuracy: 0.6167 - loss: 1.0564 - val_accuracy: 0.3980 - val_loss: 1.8412
<keras.src.callbacks.history.History at 0x7fcebb783200>
```

Initial accuracy was 40-50 not more than this



With transfer learning :

```
# transfer learning model building

# Define explicit Input layers for each branch
resnet_input = tf.keras.Input(shape=(224, 224, 3), name='resnet_input')
vgg_input = tf.keras.Input(shape=(224, 224, 3), name='vgg_input')

# Apply preprocessing using Lambda layers to the Input tensors
resnet_prepped_output = layers.Lambda(resnet_preprocess, name='resnet_prepped')(resnet_input)
vgg_prepped_output = layers.Lambda(vgg_preprocess, name='vgg_prepped')(vgg_input)

base_resnet = ResNet50(weights='imagenet', include_top=False, input_tensor=resnet_prepped_output)
base_vgg = VGG16(weights='imagenet', include_top=False, input_tensor=vgg_prepped_output)

base_resnet.trainable = False
base_vgg.trainable = False

resnet_features = base_resnet.output
vgg_features = base_vgg.output

resnet_features = GlobalAveragePooling2D()(resnet_features)
vgg_features = GlobalAveragePooling2D()(vgg_features)

combined_features = Concatenate()([resnet_features, vgg_features])

combined_features = Dense(128, activation='relu')(combined_features)
combined_features = layers.BatchNormalization()(combined_features)
combined_features = Dropout(0.5)(combined_features)
output = Dense(number_classes, activation='softmax')(combined_features)

model = Model(inputs=[resnet_input, vgg_input], outputs=output)
```

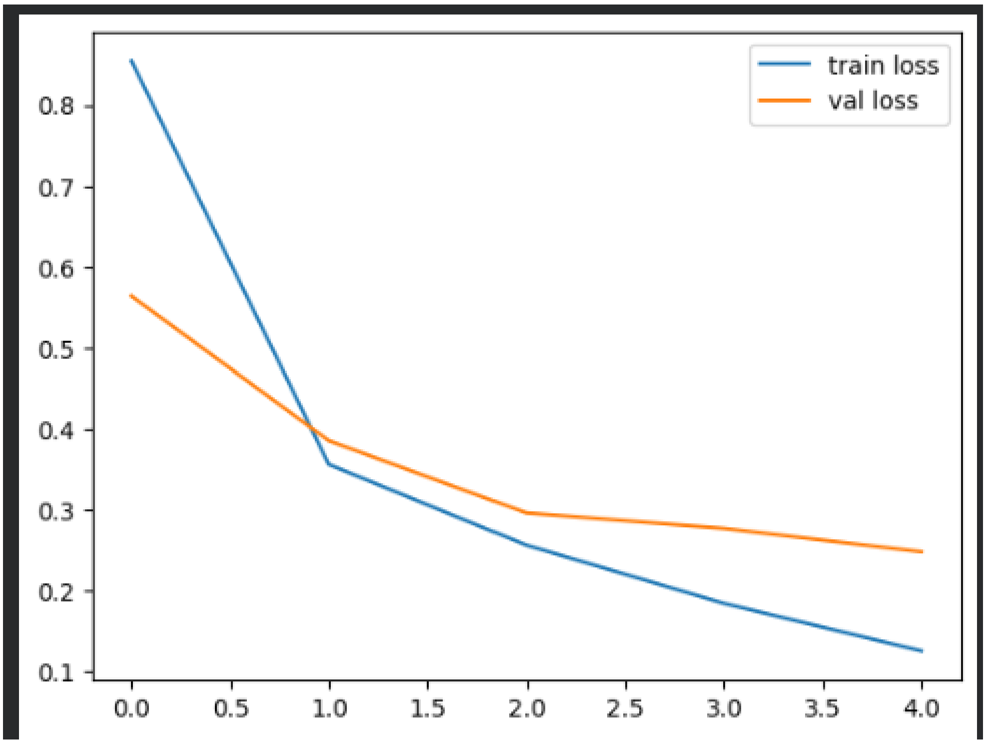
Last layers

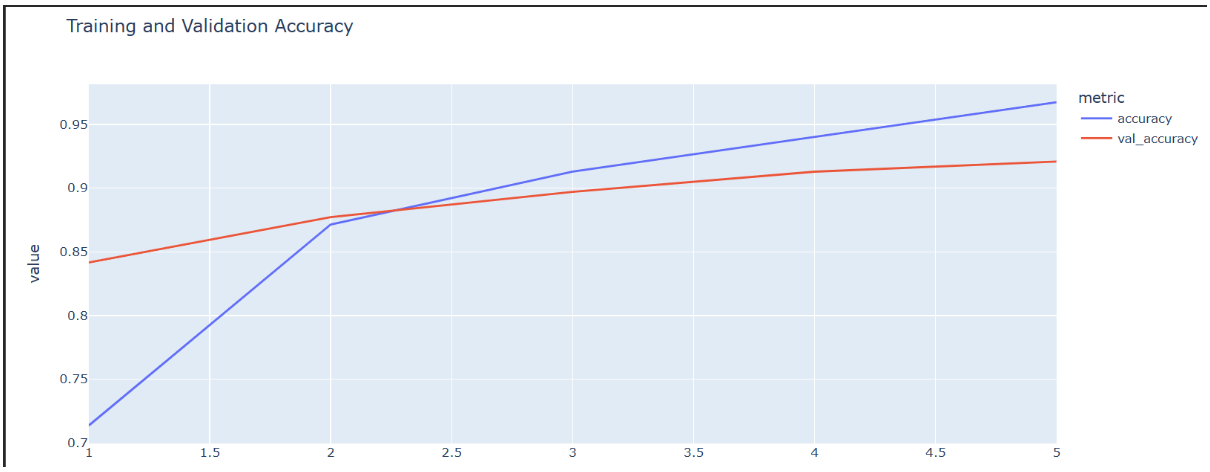
conv5_block3_add (Add)	(None, 7, 7, 2048)	0	conv5_block2_out… conv5_block3_3_b…
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808	block5_conv2[0][…
conv5_block3_out (Activation)	(None, 7, 7, 2048)	0	conv5_block3_add…
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0	block5_conv3[0][…
global_average_poo… (GlobalAveragePool…	(None, 2048)	0	conv5_block3_out…
global_average_poo… (GlobalAveragePool…	(None, 512)	0	block5_pool[0][0]
concatenate (Concatenate)	(None, 2560)	0	global_average_p… global_average_p…
dense_13 (Dense)	(None, 128)	327,808	concatenate[0][0]
dropout_8 (Dropout)	(None, 128)	0	dense_13[0][0]
dense_14 (Dense)	(None, 6)	774	dropout_8[0][0]

Total params: 38,630,982 (147.37 MB)
Trainable params: 328,582 (1.25 MB)
Non-trainable params: 38,302,400 (146.11 MB)

```
model.fit(train_ds, epochs=5, validation_data=val_ds)
```

```
Epoch 1/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 48s 560ms/step - accuracy: 0.7136 - loss: 0.8547 - val_accuracy: 0.8416 - val_loss: 0.5635
Epoch 2/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 24s 378ms/step - accuracy: 0.8714 - loss: 0.3558 - val_accuracy: 0.8772 - val_loss: 0.3852
Epoch 3/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 23s 355ms/step - accuracy: 0.9130 - loss: 0.2558 - val_accuracy: 0.8970 - val_loss: 0.2957
Epoch 4/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 23s 359ms/step - accuracy: 0.9402 - loss: 0.1837 - val_accuracy: 0.9129 - val_loss: 0.2765
Epoch 5/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 24s 372ms/step - accuracy: 0.9674 - loss: 0.1248 - val_accuracy: 0.9208 - val_loss: 0.2476
<keras.src.callbacks.history.History at 0x7fceb822c290>
```


train loss
val loss
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Training and Validation Accuracy
metric
accuracy
val_accuracy
value
0.95
0.9
0.85
0.8
0.75
0.7
1
1.5
2
2.5
3
3.5
4
4.5
5